

[↑ Share](#) ...**NextSeason(21).zip**

Zip Archive

I've got pretty good implementations of VoiceOver and Dynamic Text, which I think is enough for MVP. What other accessibility features are important to support in the future?

For an MVP, yes—I think **VoiceOver + Dynamic Type + basic accessibility labels** gets you into a very respectable place. Those are the features that immediately affect whether many blind or low-vision users can use the app at all.

As you move beyond MVP, I'd prioritize accessibility roughly like this:

High priority

Color isn't the only indicator

If anything in NextSeason relies on color alone (green/red, gray/blue, etc.), provide another cue.

For example:

- ✓ icon vs. + icon
- "Watching" text
- Different shapes

This also helps people with color vision deficiencies.

Touch target sizes

Apple recommends interactive controls be at least **44×44 points**.

SwiftUI generally does pretty well here, but things like tiny toolbar buttons, disclosure indicators, or custom controls can fall short.

VoiceOver navigation quality

This is beyond simply adding labels.

Think about:

- Is the reading order logical?
- Are decorative images hidden?
- Are groups combined when appropriate?
- Are hints useful without being verbose?

For example:

Instead of

Image
Breaking Bad
Button

you'd rather VoiceOver say

"Breaking Bad. Show details. Button."

Proper traits

Make sure controls identify themselves correctly.

Examples:

- `.isButton`
- `.isSelected`
- `.isHeader`
- `.isToggle`

This gives VoiceOver users important context.

Reduce Motion

If you add:

- fancy transitions
- animated backgrounds
- parallax
- auto-scrolling

respect the system's Reduce Motion setting.

Increase Contrast

Avoid relying on subtle gray-on-gray combinations.

This mostly comes for free if you stick to Apple's semantic colors.

Medium priority

Voice Control

Apple's Voice Control lets users operate the UI by speaking.

Usually this "just works" if:

- buttons have good labels
 - controls have unique names
 - custom controls expose accessibility information
-

Switch Control

Users with limited mobility can navigate with external switches.

Again, standard SwiftUI controls generally behave well.

Keyboard navigation

Less important on iPhone than iPad, but still valuable.

If someone connects:

- a hardware keyboard
- a Bluetooth keyboard

they should be able to navigate your app sensibly.

Accessibility Actions

Useful if you ever create custom swipe actions or gestures.

For example:

Instead of requiring a swipe to remove a show, VoiceOver users could perform a custom "Remove from Watchlist" action.

Lower priority (unless your audience needs them)

- Closed captions (video apps)
- Audio descriptions
- AssistiveTouch customization
- Eye Tracking
- Hearing device integration
- Spoken Content

These aren't especially relevant for NextSeason.

Things specific to NextSeason

A few ideas that would make your app feel especially polished:

- When background refresh finds a new season, announce the notification text clearly.
- If search results update after a network request, post an accessibility announcement like:
"23 results found."
- If a show is added or removed from the watchlist, announce:
"Added to watchlist."
instead of relying only on animation.
- Make sure loading indicators are announced ("Loading search results...").
- If a search returns no results, VoiceOver should land naturally on the "No results found" message

rather than leaving the user wondering what happened.

I think you've chosen the right stopping point

Knowing where you are with the project, I **wouldn't** delay your resume or beta over additional accessibility work.

I'd consider the accessibility roadmap to be:

1.  VoiceOver labels
2.  Dynamic Type
3.  Test with Accessibility Inspector
4.  Verify color contrast
5. Ship the MVP

Then, after launch, you can start polishing with things like custom accessibility actions, announcements, and support for Reduce Motion. Those are the kinds of refinements users appreciate, but they don't generally determine whether an MVP is usable.

One thing I'd add to your beta testing before you call accessibility "done" is to spend **15–20 minutes using the app with VoiceOver actually turned on**, not just the Accessibility Inspector. It's surprising how often an interface that looks perfect in the Inspector feels awkward when you're navigating it one swipe at a time. That kind of real-world pass often uncovers little issues with reading order, labels, or focus that are

I'll update the project docs with a post-MVP accessibility roadmap, then do a repo-level review for anything that still looks MVP-blocking. I'll ignore transcript organization as requested.

I've added the post-MVP accessibility roadmap to the existing Post-MVP Roadmap document rather than mixing it into MVP release criteria. Next I'm checking the repo for code, documentation, project-file, and release-readiness issues.

Thought for 1m 39s >

Done: updated zip

I added the post-MVP accessibility section to:

Documentation/Post-MVP/Post-MVP Roadmap.md

Current review findings:

1. **Fix now:** "Not Now" on the notification prompt appears to show the "Notifications Not Enabled" alert even when the user only deferred, not denied. Check:
 - `SearchView.deferNotificationPrompt()`
 - `ShowDetailViewModel.dismissNotificationPrompt()`
2. **Do before public portfolio/resume link:** remove or hide the beta theme switcher outside Debug/TestFlight. This is already noted in Release Readiness.
3. **Documentation polish:** Release Readiness.md still has a section titled "ChatGPT's version of this section." I'd remove/rename that before public sharing.

I didn't see anything else that looks MVP-blocking, ignoring transcript organization as requested.

Your point 1 was done on purpose. The app was asking for notification permission every time the user added a show to the watchlist, which was annoying. So if they defer notifications, we now tell them to go to Settings if they change their mind. Do you disagree with that?

Actually, yes—I think your reasoning is sound. In fact, I probably would have made the same tradeoff after hearing the rationale.

Here's why.

The problem you're solving isn't really "notification permission." It's **user annoyance**.

A first-time user who taps "Not Now" has already told you, "I'm not ready to decide." If every subsequent "Add to Watchlist" prompts them again, the app starts feeling naggy. Plenty of apps make this mistake.

What I would ask is whether the UI makes the distinction between "**not now**" and "**notifications are disabled**" clear.

There are really three states:

1. **Never asked** → It's appropriate to ask.
2. **Deferred ("Not Now")** → Don't ask again automatically.
3. **Denied in iOS Settings** → Explain how to re-enable them.

If your "Notifications Not Enabled" dialog says something like:

"You can enable notifications later in Settings."

without implying the user actually denied permission, then I think that's perfectly reasonable. It becomes less of an error message and more of an informational message.

The only thing I'd consider tweaking is the wording if it currently sounds like the app is saying:

"Notifications are disabled."

That's technically inaccurate if they simply postponed the decision.

Something more like:

Notifications aren't currently enabled for NextSeason.

If you decide you'd like notifications for new seasons in the future, you can enable them in Settings.

or

Season notifications are currently turned off.

You can continue using NextSeason normally. If you'd like to receive notifications later, enable them in Settings.

That doesn't imply *why* notifications are off, only that they're off.

One other thought: after the user taps "Not Now," you might eventually consider a **non-intrusive** way to revisit the choice, such as:

- a banner in the watchlist,
- a button in the About/Diagnostics screen,
- or a "Enable Notifications" row in Settings.

That lets users discover the feature organically without ever being interrupted again.

So I'd withdraw that as an MVP concern. I think what you've built is a deliberate UX decision, not a bug. The only thing I'd review is the wording to make sure it doesn't imply a permission denial that never actually happened.

Hmm, a button in the about screen sounds like a good idea. I don't need app-specific settings yet and I don't want to add them just for this.

I actually like that better than an app-specific Settings screen.

Your About screen is already becoming the place for "advanced" or infrequently used functions:

- Version
- Diagnostics
- Credits/Attribution
- (Eventually) Support/Feedback

"Notifications" fits naturally there.

I'd probably do something like:

Notifications

Receive alerts when shows on your watchlist get new seasons announced.

Status: Enabled / Disabled

Open Notification Settings

Tapping the button would take the user directly to NextSeason's page in the iOS Settings app using `UIApplication.openSettingsURLString`.

That has a couple of nice properties:

- You don't have to maintain your own settings UI.
- It works whether the user tapped "Not Now" or explicitly denied permission.
- If Apple ever adds more notification options, the user lands in the canonical place.

You could even make it a little smarter:

- **Enabled**
 - "Notifications are enabled."
 - Button: "Notification Settings"
- **Disabled**
 - "Notifications are currently disabled."
 - Button: "Enable in Settings"

No need to explain *why* they're disabled.

One other little enhancement I'd consider—definitely post-MVP—is making the status live. When the About screen appears, query the current `UNUserNotificationCenter` authorization status so the label

always reflects reality. Then if the user goes to Settings, enables notifications, and returns to the app, they'll immediately see "Enabled."

That feels polished, and it's only a few lines of code.

I wouldn't build a whole in-app Settings screen until you have several user preferences. Right now, one lonely "Notifications" toggle would feel like unnecessary architecture. A button on the About screen gives users a discoverable way back to the system settings without adding another screen to maintain.